

Angela Wei

Data 260

Homework 3

Section 0:

Hardware Used: HP Pavilion Windows 11 Pro

Local Model Used:

Commit Hash:

Personal Configuration and Domain

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

Main file setting up Session Middleware and connecting to authentication file. Running this file in the terminal will let the app run in the localhost on PORT_BASE 8439

```
app = FastAPI()

SECRET_KEY = os.getenv("SECRET_KEY", "dev-only-secret-key")

app.add_middleware(
    SessionMiddleware,
    secret_key=SECRET_KEY,
    https_only=True,
    same_site="lax",
    max_age=3600
)

#connect the router for hw3_authentication.py to work upon running the file
app.include_router(auth_router)

#use to run the app on PORT_BASE 8439
if __name__ == "__main__":
    uvicorn.run(
        "hw3_main:app",
        host="127.0.0.1",
        port=8439,
        reload=True
    )
```

After logging in with valid credentials, the session is set to automatically timeout after 5 minutes or 300 seconds of inactivity. Becomes active once a session starts after login. A timeout should end/clear a session upon happening.

```
#timeout after 5 minutes/300 seconds of inactivity
idle_timeout_seconds = 300

def idle_timeout(request: Request):
    last_activity = request.session.get("last_activity")
    current_time = time.time()

    if last_activity:
        if current_time - last_activity > idle_timeout_seconds:
            request.session.clear()
            return True

    if "user" in request.session:
        request.session["last_activity"] = current_time

    return False
```

All valid credentials (username:password) are stored as a dictionary database.

```
#dictionary database containing username followed by corresponding password
#Username Password
user_data = {"AnW_123": "f_password12345",
            "HaW_246": "s_password24680",
            "Wei0602": "g_password35912",
            "MSADI_27": "sjsu_word4814"}
```

Here, the login_page function connects with the login HTML page in the templates file. It should show up in the localhost when running. Should also show up again after a session timeout.

```
#get the login page
@router.get("/login")
def login_page(request: Request, error: str = None):
    #start the session
    user = request.session.get("user")

    #stay on the dashboard as long as there is no timeout
    if user and not idle_timeout(request):
        return RedirectResponse(
            url = "/dashboard",
            status_code=HTTP_302_FOUND
        )

    #display the login page
    return templates.TemplateResponse(
        request = request,
        name = "hw3_login.html",
        context = {
            "user": user,
            "error": error
        }
    )
```

The `get_login` function should retrieve the login credentials that were entered and redirect to user's dashboard if the credentials are valid. Should keep user on the login page if invalid credentials are entered.

```
#get user login input
@router.post("/login")
def get_login(request: Request, user: str = Form(...), pass_wrd: str = Form(...)):
    get_cred = user_data.get(user)

    if get_cred and get_cred == pass_wrd:
        #if the user enters a valid username and password
        #aka if the credentials already exist
        request.session["user"] = user
        request.session["last_activity"] = time.time()
        return RedirectResponse(
            #can go to user dashboard if valid
            url = "/dashboard",
            status_code=HTTP_302_FOUND
        )

    #otherwise stay on the login page
    #if there are no valid credentials
    return RedirectResponse(
        url = "/login?error=Invalid+username+or+password",
        status_code=HTTP_302_FOUND
    )
```

User's dashboard should display if valid credentials are entered. Here a session starts. Should redirect back to login if a session timeout happens.

```
#display user dashboard after logging in
@router.get("/dashboard")
def dashboard_page(request: Request):
    #go back to login page if inactivity timeout hits
    if idle_timeout(request):
        return RedirectResponse(
            url = "/login",
            status_code = HTTP_302_FOUND
        )

    #go to the dashboard page if the user is approved
    user = request.session.get("user")

    #do not display the dashboard if the user is not logged in
    if not user:
        return RedirectResponse(
            url = "/login",
            status_code = HTTP_302_FOUND
        )

    #display the dashboard if user is logged in
    return templates.TemplateResponse(
        request = request,
        name = "hw3_dashboard.html",
        context = {
            "user": user
        }
    )
```

If a user selects the Log Out button, the session should end/clear automatically.

```
#logout out of users account
@router.get("/logout")
def log_out_account(request: Request):

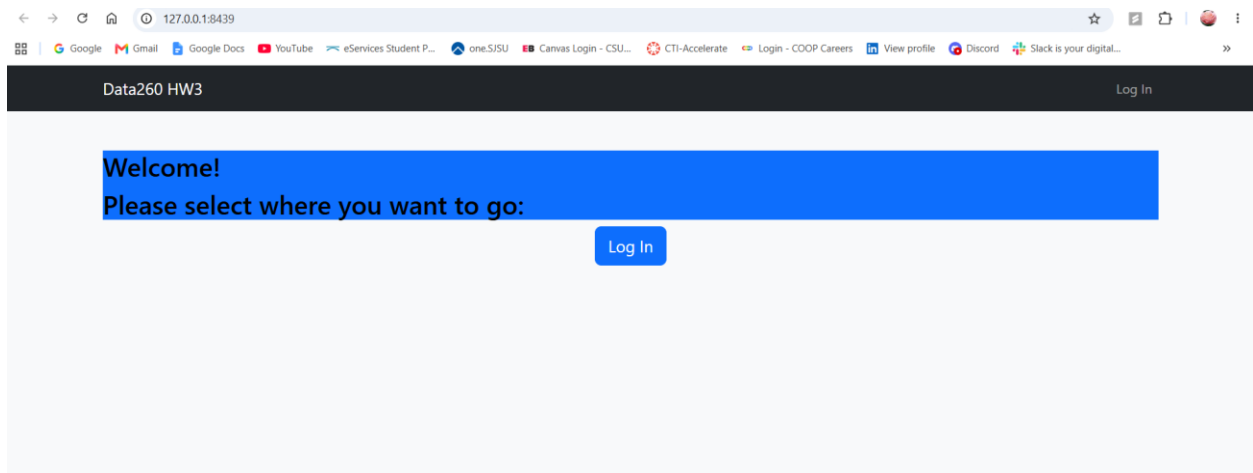
    #end the session upon logging out of user account
    request.session.clear()

    #redirect the user back to the homepage
    return RedirectResponse(
        url="/",
        status_code=HTTP_302_FOUND
    )
```

HTML file showing the Welcome page. Will display a Log In button if user is not logged into account. Will display a Dashboard and Log Out button if user is logged in.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class = "bg-light">
    <nav class = "navbar navbar-expand-lg navbar-dark bg-dark mb-4">
        <div class = "container">
            <a class = "navbar-brand" href = "/">Data260 HW3</a>
            <div class = "navbar-nav ms-auto">
                {% if user %}
                <span class = "nav-link text-light me-3">Logged in as: {{user}} </span>
                <a class = "nav-link" href = "/logout">Log Out</a>
                {% else %}
                <a class = "nav-link" href = "/login">Log In</a>
                {% endif %}
            </div>
        </div>
    </nav>
    <div class = "container mt-5">
        <div class = "card-header bg-primary text-black">
            <h2>Welcome!</h2>
            <h2>Please select where you want to go:</h2>
        </div>
        <div class = "card-body p-6 text-center">
            {% if user %}
            <div class = "mb-3">
                <a href = "/dashboard" class = "btn btn-primary btn-lg" >Go to Dashboard</a>
                <a href = "/logout" class = "btn btn-danger btn-lg" >Log Out</a>
            </div>
            {% else %}
            <div class = "mb-3">
                <a href = "/login" class = "btn btn-primary btn-lg" >Log In</a>
            </div>
            {% endif %}
        </div>
    </div>
</div>
```

Website running on PORT_BASE in localhost. Home/Welcome page when user is not logged in.



HTML file for Log In page.

```
<head>
<title>Log Into Your Account</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class = "bg-light">
  <div class = "container mt-5">
    {% if error %}
    <div class = "alert alert-danger alert-dismissible fade show" role = "alert">
      <strong>Log in failed!</strong>{{error}}
      <button type = "button" class = "btn-close" data-bs-dismiss = "alert" aria-label = "Close"></button>
    </div>
    {% endif %}
    <div class = "card-header bg-primary text-black">
      <h2>Log In</h2>
    </div>

    <div class = "card-body p-6">
      <form action = "/login" method = "post">
        <div class = "mb-3">
          <label for = "user" class = "form-label">Username * :</label><br>
          <input type = "text" id = "user" name = "user" class = "form-control" placeholder = "Username" required>
        </div>

        <div class = "mb-3">
          <label for = "pass" class = "form-label">Password * :</label><br>
          <input type = "text" id = "pass" name = "pass_wrd" class = "form-control" placeholder = "Password" required>
        </div>
        <div class = "d-grid gap-3">
          <button type = "submit" class = "btn btn-primary">Submit</button>
        </div>
      </form>
    </div>
  </div>
</body>
```

Log In page displaying error message after user entered invalid credentials.

Log in failed!Invalid username or password

Log In

Username * :

Username

Password * :

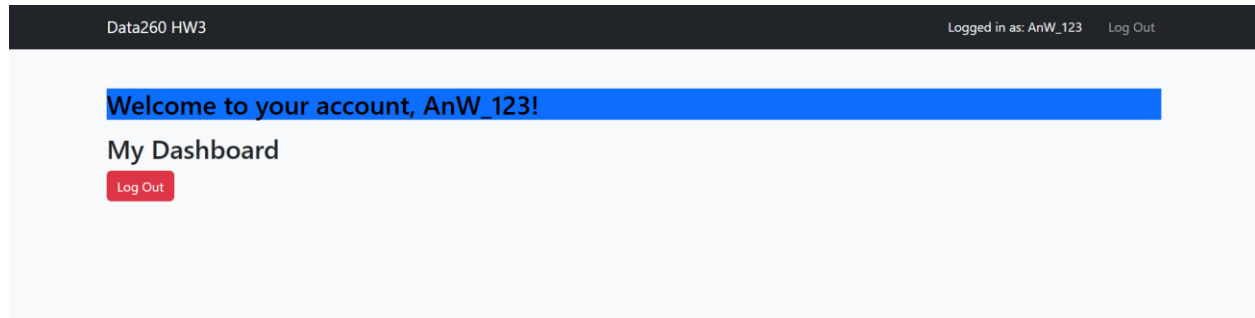
Password

Submit

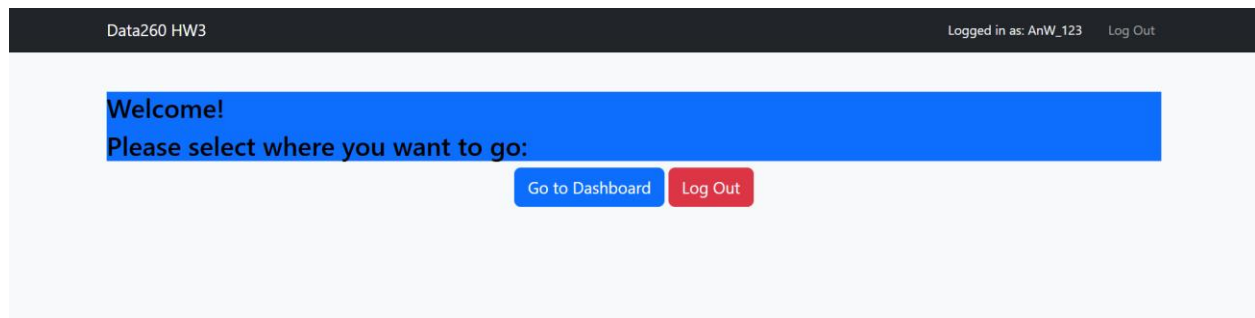
HTML file for the user's Dashboard. Indicated which user is currently logged in. Will let user log out of account manually if they are done.

```
<!DOCTYPE html>
<html lang = "en">
<head>
<title>My Dashboard</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class = "bg-light">
  <nav class = "navbar navbar-expand-lg navbar-dark bg-dark mb-4">
    <div class = "container">
      <a class = "navbar-brand" href = "/">Data260 HW3</a>
      <div class = "navbar-nav ms-auto">
        <span class = "nav-link text-light me-3">Logged in as: {{user}} </span>
        <a class = "nav-link" href = "/logout">Log Out</a>
      </div>
    </div>
  </nav>
  <div class = "container mt-5">
    <div class = "card-header bg-primary text-black">
      <h2 class = "card-title mb-3">Welcome to your account, {{ user }}!</h2>
    </div>
    <div class = "card-body p-6">
      <form>
        <div class = "mb-3">
          <h2>My Dashboard</h2>
          <a href = "/logout" class = "btn btn-danger">Log Out</a>
        </div>
      </form>
    </div>
  </div>
</body>
</html>
```

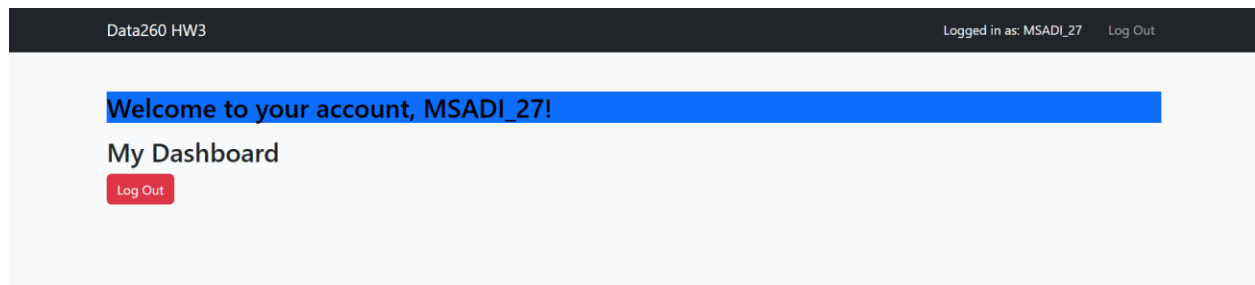
User dashboard after logging in.



Welcome/Home page when the user is logged in. Notice that Dashboard is only accessible from here.



Different user logged onto their dashboard.



Terminal during an active session.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python hw3_main.py
INFO: Will watch for changes in these directories: ['C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code']
INFO: Uvicorn running on http://127.0.0.1:8439 (Press CTRL+C to quit)
INFO: Started reloader process [19564] using StatReload
INFO: Started server process [33956]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:59069 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:59069 - "GET /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:60634 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:60634 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:60634 - "GET /logout HTTP/1.1" 302 Found
INFO: 127.0.0.1:60634 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:60634 - "GET /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:57455 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:57455 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:55366 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:55366 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:55366 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:52197 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:50503 - "GET /logout HTTP/1.1" 302 Found
INFO: 127.0.0.1:50503 - "GET / HTTP/1.1" 200 OK
```

Part 2

Downloaded all the required packages through a single command.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> pip install --user llama-index llama-index-embeddings-huggingface sentence-transformers faiss-cpu numpy pandas
WARNING: Ignoring invalid distribution ~ (C:\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~-p (C:\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (C:\Python312\Lib\site-packages)
WARNING: Ignoring invalid distribution ~-p (C:\Python312\Lib\site-packages)
```

```
Successfully installed aiosqlite-0.22.1 banks-2.5.1 click-8.5.0 dataclasses-json-0.6.7 defusedxml-0.7.1 deprecated-1.3.1 dirtyjson-1.0.8 faiss-cpu-1.15.1 fsspec-2026.9.0 griffe-2.3.0 griffecli-2.3.0 griffelib-2.3.0 hf-xet-1.6.0 huggingface-hub-1.32.0 jiter-0.17.0 llama-index-0.14.24 llama-index-core-0.14.24 llama-index-embeddings-huggingface-0.8.0 llama-index-embeddings-openai-0.6.0 llama-index-instrumentation-0.6.0 llama-index-llms-openai-0.7.10 llama-index-workflows-2.24.1 markdown-it-py-4.2.0 marshmallow-3.26.2 mdurl-0.1.2 mypy-extensions-1.1.0 nltk-3.10.3 openai-2.54.0 pillow-12.3.0 pygmen-ts-2.21.0 regex-2026.9.10 rich-15.0.0 safetensors-0.8.0 sentence-transformers-6.1.0 shellingham-1.5.4 sympy-1.14.0 tiktoken-0.14.0 tinytag-2.3.2 tokenizers-0.23.2 torch-2.14.0 tqdm-4.70.1 transformers-5.17.0 typer-0.27.2 typing-inspect-0.9.0 wrapt-2.4.1
```


My DOMAIN_ID is grocery supply and recall notices. I used a dataset by the FDA on Food Enforcement Reports

The screenshot shows the openFDA website interface. The top navigation bar includes the FDA logo, 'U.S. FOOD & DRUG ADMINISTRATION', and the U.S. Department of Health and Human Services Food and Drug Administration. The main navigation menu has links for Home, About, APIs (highlighted), Data, Community, and Disclaimer. On the left, a sidebar lists various API categories: API Basics, Construct the Query, Download the Data, Animal & Veterinary API Endpoints, Drug API Endpoints, Device API Endpoints, and Food API Endpoints. Under Food API Endpoints, 'Recall Enforcement Reports' is selected, showing sub-links for Overview, How to use the API, Example API queries, and Explore the API with an interactive chart. The main content area is titled 'Download the dataset' and provides instructions on how to download the dataset manually or via API. It specifically mentions the 'Food Recall Enforcement Reports [/food/enforcement]' endpoint, noting that data is in zipped JSON files and updates may change old records. It also states there are 1 file, last updated on 2026-09-16, and provides a link to '/food/enforcement data 5.58 mb'.

Retrieving information from the database URL for chunking.

```
#Getting FDA food recall data
def load_openfda_documents(limit: int = 50) -> List[Document]:
    url = f"https://api.fda.gov/food/enforcement.json?limit={limit}"
    print(f"Fetching data from openFDA API {limit}")

    response = requests.get(url)
    if response.status_code != 200:
        raise RuntimeError(f"Failed to fetch the data")

    data = response.json()
    results = data.get("results", [])

    documents = []
    for record in results:
        #creating text content chunking
        text = (
            f"Recalling Firm: {record.get('recalling_firm', 'N/A')}\n"
            f"Product Description: {record.get('product_description', 'N/A')}\n"
            f"Reason for Recall: {record.get('reason_for_recall', 'N/A')}\n"
            f"Classification: {record.get('classification', 'N/A')}\n"
            f"Status: {record.get('status', 'N/A')}\n"
        )

        #store metadata for traceability
        metadata = {
            "recall_number": record.get("recall_number", ""),
            "recalling_firm": record.get("recalling_firm", ""),
            "classification": record.get("classification", "")
        }

        documents.append(Document(text=text, extra_info=metadata))
```

Functions to get the cosine similarities and chunking summaries

```
def cosine_similarity(v1:list, v2:list) -> float:
    a = np.array(v1)
    b = np.array(v2)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def get_nodes(technique: str, docs: List[Document]):
    if technique == "Token":
        splitter = TokenTextSplitter(chunk_size=50, chunk_overlap=10)
        return splitter.get_nodes_from_documents(docs)

    elif technique == "Semantic":
        splitter = SemanticSplitterNodeParser(
            buffer_size=1,
            breakpoint_percentile_threshold=95,
            embed_model=embed_model
        )
        return splitter.get_nodes_from_documents(docs)

    elif technique == "Sentence-window":
        splitter = SentenceWindowNodeParser.from_defaults(
            window_size=3,
            window_metadata_key="window",
            original_text_metadata_key="original_sentence"
        )
        return splitter.get_nodes_from_documents(docs)

    else:
        raise ValueError(f"Unknown technique")
```

Function to retrieve all information

```
def run_retrieval_pipeline(query: str, technique: str, docs: List[Document], k: int = 3):

    #chunking and vector store index
    nodes = get_nodes(technique, docs)
    index = VectorStoreIndex(nodes)
    retriever = index.as_retriever(similarity_top_k=k)

    #query embedding
    query_embedding = embed_model.get_query_embedding(query)
    print(f"Query Embedding Dimension: {len(query_embedding)}")
    print(f"First 8 Query Embedding Values: {[round(x, 4) for x in query_embedding[:8]]}\n")

    #retrieve the nodes
    retrieved_nodes = retriever.retrieve(query)
```

Results from running the file:

```
Query Embedding Dimension: 384
First 8 Query Embedding Values: [-0.0272, -0.0274, 0.0698, 0.0464, 0.0843, 0.0345, 0.0338, 0.0399]

Query Vector Shape: (384,)
Stacked Doc Vectors Shape: (3, 384)

rank  store_score  cosine_sim  chunk_len
preview
1      0.6731      0.7850      67      - Food Service Reason for Recall: Undeclared allergens: whe
at, soy,
2      0.6265      0.5298      54      Undeclared allergens: wheat, soy, crustacean s
hellfish
3      0.6253      0.5343      86 the product contains undeclared allergens. Classification: Class II Status: Te
rminated

Saved raw data to: raw/retrieval_token.json
```

```
Query Embedding Dimension: 384
First 8 Query Embedding Values: [-0.0272, -0.0274, 0.0698, 0.0464, 0.0843, 0.0345, 0.0338, 0.0399]

Query Vector Shape: (384,)
Stacked Doc Vectors Shape: (3, 384)

rank  store_score  cosine_sim  chunk_len
preview
1      0.6292      0.6817      165 By: Trophy Nut Co. Tipp City, OH 45371 Reason for Recall: The firm stated that
the product contains undeclared allergens. Classification: Class II Status: Termi
2      0.5897      0.5577      749 Recalling Firm: Oasis Brands, Inc Product Description: Crema Guatelinda (Guate
malan Style Cream) in individually soft poly/plastic bags labeled in part: Lacteos
3      0.5871      0.6027      262 -Reser's Gourmet Tuna Salad packaged in 2/5-lb carton cases. Reser's Fine Foo
ds, Inc., Beaverton, OR. UPC 071117855067 Reason for Recall: The recalled product

Saved raw data to: raw/retrieval_semantic.json
```

```
Query Embedding Dimension: 384
First 8 Query Embedding Values: [-0.0272, -0.0274, 0.0698, 0.0464, 0.0843, 0.0345, 0.0338, 0.0399]

Query Vector Shape: (384,)
Stacked Doc Vectors Shape: (3, 384)

rank  store_score  cosine_sim  chunk_len
preview
1      0.5989      0.5433      516 Recalling Firm: Oasis Brands, Inc Product Description: Crema Guatelinda (Guate
malan Style Cream) in individually soft poly/plastic bags labeled in part: Lacteos
2      0.5825      0.6924      103      Tipp City, OH 45371 R
eason for Recall: The firm stated that the product contains undeclared allergens.
3      0.5804      0.5355      218 Product Description: "Alcapurria Jueyes" (Crab-filled fritters) - Food Service
Reason for Recall: Undeclared allergens: wheat, soy, crustacean shellfish (crab),

Saved raw data to: raw/retrieval_sentence_window.json
```

Token output:

```
{
  "technique": "Token",
  "query": "What food recalls are related to contamination or undeclared allergens?",
  "query_vector_shape": [
    384
  ],
  "doc_vectors_shape": [
    3,
    384
  ],
  "results": [
    {
      "rank": 1,
      "store_score": 0.6731,
      "cosine_sim": 0.785,
      "chunk_len": 67,
      "preview": "- Food Service Reason for Recall: Undeclared allergens: wheat, soy,"
    },
    {
      "rank": 2,
      "store_score": 0.6265,
      "cosine_sim": 0.5298,
      "chunk_len": 54,
      "preview": "Undeclared allergens: wheat, soy, crustacean shellfish"
    },
    {
      "rank": 3,
      "store_score": 0.6253,
      "cosine_sim": 0.5343,
      "chunk_len": 86,
      "preview": "the product contains undeclared allergens. Classification: Class II Status: Terminated"
    }
  ]
}
```

Semantic output:

```
{
  "technique": "Semantic",
  "query": "What food recalls are related to contamination or undeclared allergens?",
  "query_vector_shape": [
    384
  ],
  "doc_vectors_shape": [
    3,
    384
  ],
  "results": [
    {
      "rank": 1,
      "store_score": 0.6292,
      "cosine_sim": 0.6817,
      "chunk_len": 165,
      "preview": "By: Trophy Nut Co. Tipp City, OH 45371 Reason for Recall: The firm stated that the product contains undeclared allergens. Classification: Class II Stat"
    },
    {
      "rank": 2,
      "store_score": 0.5897,
      "cosine_sim": 0.5577,
      "chunk_len": 749,
      "preview": "Recalling Firm: Oasis Brands, Inc Product Description: Crema Guatelinda (Guatemalan Style Cream) in individually soft poly/plastic bags labeled in part"
    },
    {
      "rank": 3,
      "store_score": 0.5871,
      "cosine_sim": 0.6027,
      "chunk_len": 262,
      "preview": "-Reser's Gourmet Tuna Salad packaged in 2/5-lb carton cases. Reser's Fine Foods, Inc., Beaverton, OR. UPC 071117855067 Reason for Recall: The recalle"
    }
  ]
}
```

Sentence window output:

```
{
  "technique": "Sentence-window",
  "query": "what food recalls are related to contamination or undeclared allergens?",
  "query_vector_shape": [
    384
  ],
  "doc_vectors_shape": [
    3,
    384
  ],
  "results": [
    {
      "rank": 1,
      "store_score": 0.5989,
      "cosine_sim": 0.5433,
      "chunk_len": 516,
      "preview": "Recalling Firm: Oasis Brands, Inc Product Description: Crema Guatelinda (Guatemalan Style Cream) in individually soft poly/plastic bags labeled in part"
    },
    {
      "rank": 2,
      "store_score": 0.5825,
      "cosine_sim": 0.6924,
      "chunk_len": 103,
      "preview": "Tipp City, OH 45371 Reason for Recall: The firm stated that the product contains undeclared allergens. "
    },
    {
      "rank": 3,
      "store_score": 0.5804,
      "cosine_sim": 0.5355,
      "chunk_len": 218,
      "preview": "Product Description: \"Alcapurria Jueyes\" (Crab-filled fritters) - Food Service Reason for Recall: Undeclared allergens: wheat, soy, crustacean shellf"
    }
  ]
}
```

Looking at the results of the three techniques that were applied, Tokens had the shortest average chunk length, followed by Sentence Window and finally Semantic at the longest. This outcome is likely due to what the focus of each technique is. Since Sentence Window and Semantic both focus on full length sentences it should not be surprising that chunk lengths can be at least 100 characters long. A token can be as short as one word, a part of a longer word or even a single letter.

The technique I believe would be the best for this corpus would be Semantic chunking. This technique does not split off mid-sentence or mid word like Token chunking, and can cover sentences on a specific topic without having to use a large amount of time like Sentence Window.

Github:

<https://github.com/Aywei2001/data260--SID4->